Warsaw University of Technology
Faculty of Mathematics
and Information Science

Bachelor's diploma thesis

in the field of study Computer Science and Information Systems

Design and Implementation of a Collaborative Board Using the Local-First Approach

Piotr Jankiewicz

student record book number 288767

thesis supervisor

dr inż. Paweł Kotowski

WARSAW 2026

Abstract

Design and Implementation of a Collaborative Board Using the Local-First Approach

The purpose of this thesis is to design and implement a collaborative board using the local-first approach. The solution uses distribuited conflict free replicated data types (CRDTs) and real-time communication via WebRTC protocol to bring effortless collaboration with other users.
Final result of the effort done is an desktop application that enables drawing and erasing on the whiteboard with mutiple users simultaneously. This paper covers description of solution, but also thouches broader topic of local-first application design architectures.

Key words: rust language, local-first, CRDT, WebRTC, distributed system, star topology, multithread real time communication

Streszczenie

Projekt i wdrożenie kolaboratywnej tablicy z wykorzystaniem podejścia „local-first"

Celem pracy inynierskiej jest zaprojektowanie i implementacja kolaboratywnej tablicy z wykorzystaniem podejścia "local-first". Wypracowane rozwiązanie uzywa rozproszonego bezkonfliktowego typy danych (CRDTs) oraz komunikacji w czasie rzeczywistym przez protokół WebrRTC by umozliwic bezproblemową kolaborację z innymi uzytkownikami. Ostatecznym wynikiem jest aplikacja desktop-owa umozliwiająca rysowanie oraz zmazywanie poprzednich pociągnięć na białej tablicy z wieloma uzytkownikami jednocześnie. Praca pokrywa opis rozwiązania ale równiez porusza bardziej obszerny temat architektur typu local-first w aplikacjach kolaboratywnych.
Słowa kluczowe: język rust, local-first, CRDT, WebRTC, system rozproszony, topologia gwiazdy, wielowątkowa komunikacja w czasie rzeczywistym

Contents

1. Introduction **11**

Introduction **11**

1.1. Analysis and description of collaborative local-first applications 11

1.2. Local-first vs Cloud-first 15

1.3. Why would one implement own collaborative system ? 17

1.4. Examples from industry 17

1.5. Known architectures for collaborative applications 17

1.6. OT vs CRDTs 18

1.7. Vision of the System 19

1.8. Requirements Specification 19

1.8.1. Functional Requirements 19

1.8.2. Non-Functional Requirements 24

1.9. Business Cases 26

1.10. User Interface vision 26

2. Description of the solution **28**

2.1. System Architecture 28

2.2. Technology Stack 28

2.3. Application Modules 30

2.3.1. User Interface Modules and Implementation 30

2.3.2. Backend Module 32

2.4. Synchronization Protocol – How consistency is achieved 34

2.4.1. State-based vs Operation-based synchronization 34

2.4.2. Automerge sync protocol 36

2.4.3. Conflict resolution and guarantees 38

2.4.4. Initial synchronization of a new peer 39

2.5. CRDT Data Model – What is being synchronized? 40

2.5.1. Document schema 40

2.5.2. Stroke representation 41
2.5.3. Intent abstraction 42
2.5.4. File persistence 44
2.6. Communication Layer - Where the messages travel ? 45
2.6.1. LiveKit and WebRTC data channels 45
2.6.2. Thread modeling 46
2.6.3. Message flow and chunking 47
2.6.4. Cursor synchronization 48
3. Analysis of the solution 49
4. Conclusions 50

1. Introduction

1.1. Analysis and description of collaborative local-first applications

Collaborative applications have not always been so popular as they are these days. You might not even know that some of applications that we use every day are collaborative. In everyday life of a software developer almost all of modern IDEs have collaborative work options, almost all project management software are also collaborative. These are examples close to software engineers, but these applications do not limit themselves to software development. Google Docs, Microsoft 365, Figma, Miro and many more are examples of collaborative applications that are used by millions of people every day. These applications are not only used for work, but also for education and entertainment.

It hasn't always been like this. The steep rise of collaborative applications began as the remote work became more popular but also it owes its rise of popularity to universality of fast global network and increase of computing power of everyday devices.

Current collaborative apps couldn't exist 20 years ago. They haven't been simply invented yet. Typical application of early days of online applications had a simple architecture. It was a client-server application. The server was responsible for storing data and processing it, while the client was responsible for presenting the data to the user and sending user input to the server. This architecture had its advantages and disadvantages. It was simple and easy to implement, but it had a single point of failure. If the server went down, the entire application went down. It also had scalability issues, as the server had to handle all the requests from the clients.

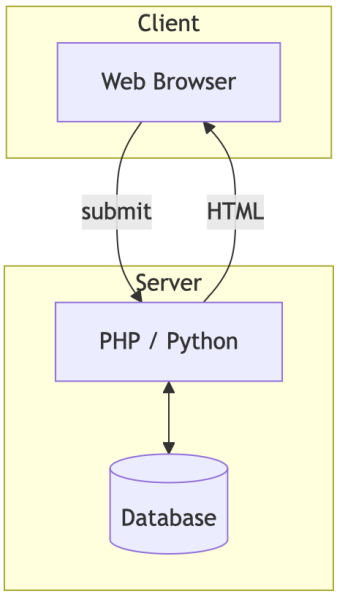

Figure 1.1: Simple client-server architecture of early internet era applications

These kind of application belong to "cloud-first" family. There was no need for "local-first" architectures yet and this term has not been popularized. Before even the discussion on topic of "local-first architectures" the "cloud-first" approach had its own more than one decade of fast development. From simple client server architectures the system rose to more difficult forms of large size client-side application with lots of layers and microservices. Picture below shows architectures that emerged on the market with rise of popularity of mobile and web apps from 2010.

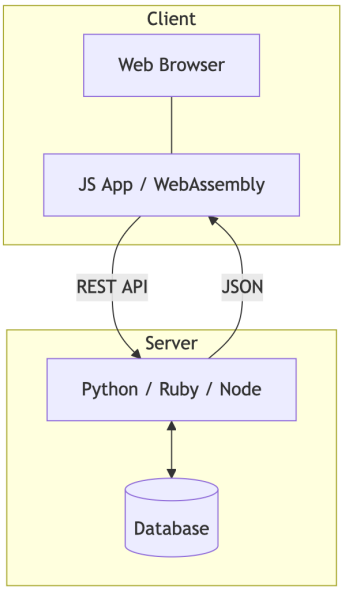

Figure 1.2: More complicated architecture of modern mobile/web applications

How far are these architectures from beeing good fit for collaboarative appliactions? Lets have a look on typical modules that take part in standrd user interface interaction. Picure below shows typical modules of morden web/mobile/desktop cloud-application.

Figure 1.3: Typical modules of modern cloud-based applications

As we can see the level of complication if very high. Simple user intercation with a system triggers multiple functions from multiple modules from multiulpe services. This level of sophistication simply cannot be used for collaborative applications, because orchestration of synchoronization of state shown in simultaneus users would be extremely difficult.

For one, in would require to many asynchronous time-consuming remote calls to different services. Total synchronization nightmare. For some time tough there has been efforts to implement collaboration this way. This approach was known as OT - operational transformation (still used in Google Docs). Although Google Docs is a good example of usage of cloud-first approach, in this thesis I decided to focus on local-first architecutres as they have proved to be more broadly used in the industry and it offers way more possibilities of different kind of applications.

There are also my other reasons to avoid cloud-first architectures when designing collaborative applications. Lets take a look at simple sequance of user interaction with the system. Below we can see general asynchronous interaction of a user with normal cloud-basen application.

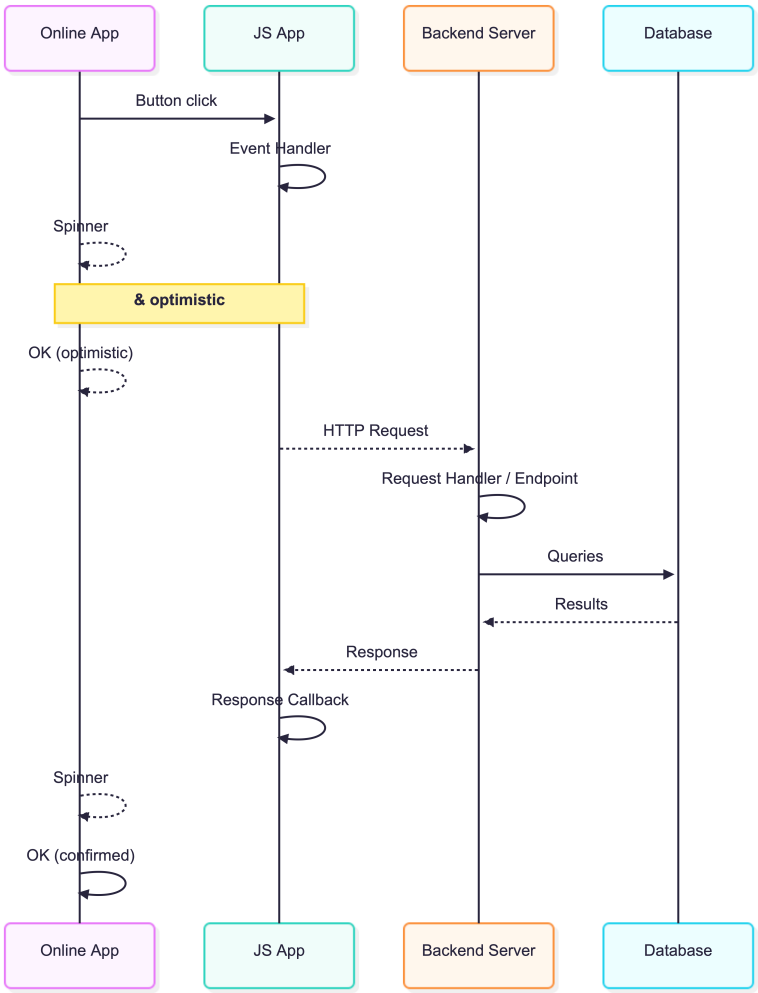

Figure 1.4: Typical sequance of interaction with a user in a modern cloud-based applications

As we can see on on this diagram, the system relies heavily on connectivity with the database. The big question that comes comes to mind is - what happens to the system if there is no internet ? Answer is simple. It would not work. Therefore, one could even say, that whole application is just a asynchronous graphical database wrapper, and it wouldn't be far from truth. This brings us the quastion - What is the definition of cloud-first and local first applications?

In simplest words the definition of cloud-approach is "If the interaction of the user with the system haven't been saved to database, it never happened". On other hand local-first is "What's important the most is local persistance - cloud brings only connectivity".

Therefore let's have a look on how would typical sequance of user interaction with local-first system look. The diagram below shows typical sequance in such system.

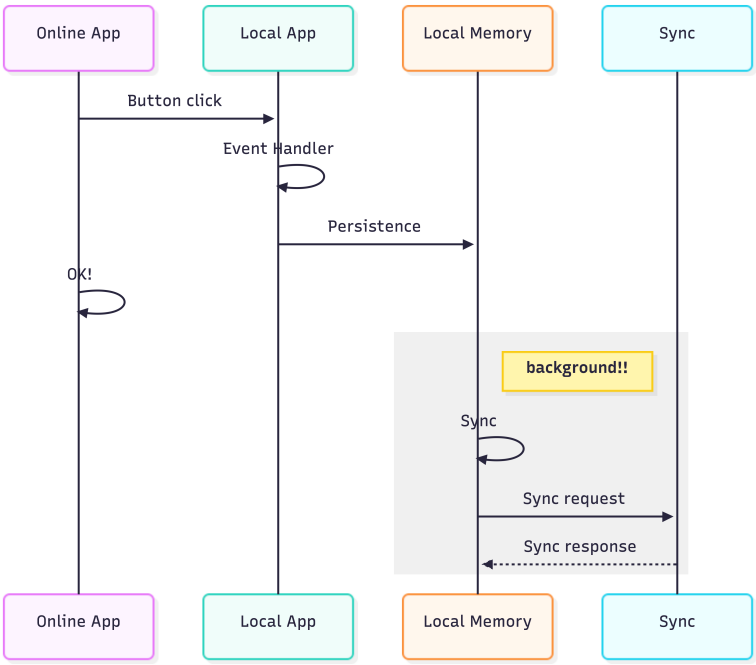

Figure 1.5: Typical sequance of interaction with a user in a local-first applications

The distinction is clear. In local-first approach the user interaction with the system is not dependent on connectivity with the database. Connectivity brings only messaging with other peers. Whole synchronization of replicas of data between users happens on local machine. This is the reaseon why we can call this system distributed. We can also notice that this approach requires much less modules across different serivces. Later, in the thesis, we will clearly how this concept allows us to bring true real time collaboration to the users.

1.2. Local-first vs Cloud-first

Lets have a look on the main differences between local-first and cloud-first approaches. The table below shows the main differences between these two approaches when it comes to designing collaborative application.

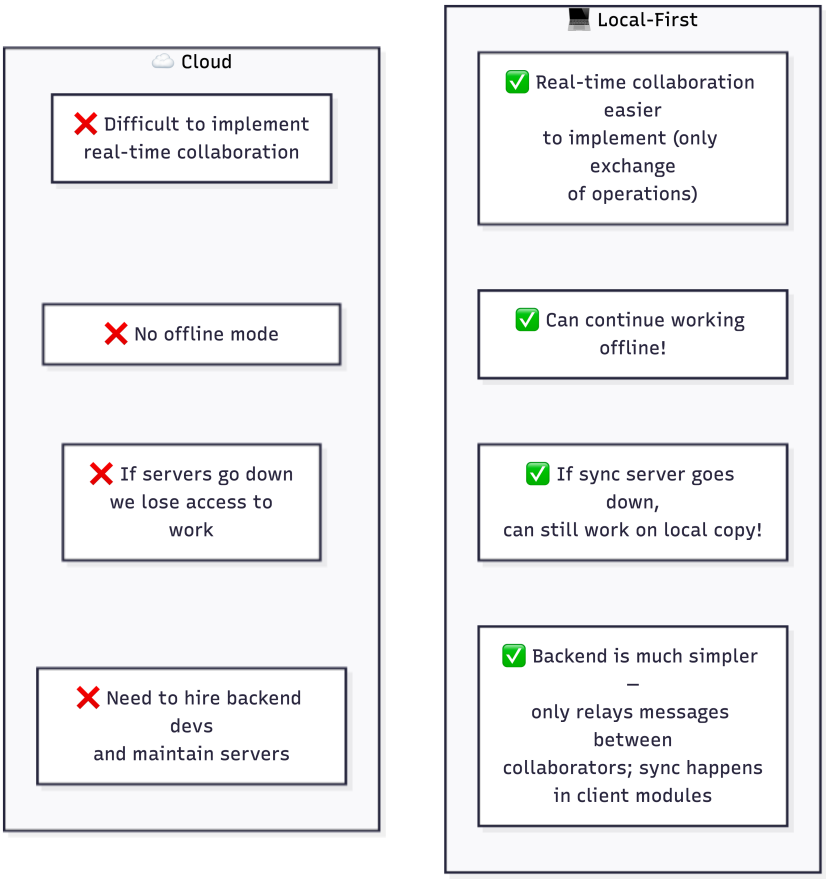

Figure 1.6: Cloud vs Local comparison

Clearly, the obvious choice for apprach when designing collaborative application would be Local-first. Let's now try to distinguish, what are boundaries of such system - what use cases are not applicable with local-first apprach. The table below shows use cases that are not good fit for local-first approach.

Table 1.1: Use cases: Suitable and Unsuitable for Local-First Approach

Suitable for	**Unsuitable for**
Real-time collaborative editing (documents, whiteboards)	Applications requiring strong centralized control
Offline-first mobile/desktop apps	Systems with strict regulatory or audit requirements
Peer-to-peer file sharing	Applications with massive global data aggregation
Personal productivity tools with sync	Centralized transactional systems (e.g., banking)
Small/medium team collaboration	Apps needing immediate global consistency

1.3. Why would one implement own collaborative system ?

Creating collaborative whiterboard is just an intriduction to the general topic of distrubuted conflict free real time data storage with editable operations. Designign whiteboard is easy exaple for system that can be used to many use cases (distribited conflict free data) - it serves as a great introduction to this topic.

There are multiple use cases/possible usage of distrubuted systems. For example drone swarms - distributed replicas of formation among drones broadcasted between each other. Peer-to-peer file sharing - distributed replicas of files among users. Real time collaborative editing - distributed replicas of documents among users.

1.4. Examples from industry

Known examples of local-first applications are: Figma, Miro, Notion, Obsidian, Roam Research, Coda, Google Docs (although it is not pure local-first application). These applications are used by millions of people every day. They are used for work, education and entertainment.

1.5. Known architectures for collaborative applications

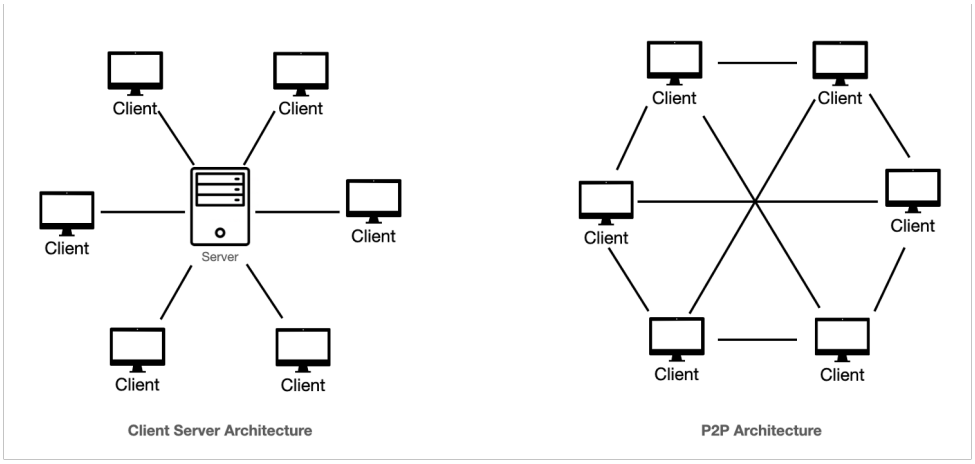

Figure 1.7: Comparison of Client-Server and Peer-to-Peer (P2P) Architectures

Connectivity among multiple users is always solved eigher by peer-to-peer or more "centered" relay architecure for example start-topology. When designig a collaborative system each solution has its benefits and setbacks. In a table belowe there is simple analysis of pros and cons of

choosing one or the ohter when designing local-first collaborative whiteboard.

Table 1.2: Comparison of Peer-to-Peer and Star Topology Architectures

Peer-to-Peer (P2P)	**Star Topology**
No need to create a server for communication. Communication is direct - good. Clients manage their connections themselves - might be problematic.	Central node relays communication. It requires support of server. Keeping servers running is expansive.
Communication might be faster because it is direct. Users might be close to each other.	Clients sometimes might unnesesarry communicate via very distant server to both of them, which would cause delays.
High resilience to single node failure	Central node is a single point of failure
Complex synchronization and conflict resolution - corrdination of connections has to be implemented on client side.	Easier management and coordination - server keeps all informations on connectivity. Clients connect only to one server.
Scalability can be challenging - many users might have very low maximum of connections it could sustain.	Scalability depends on central node capacity, but server can easily be scaled hirizontlay or verticaly
Suitable for decentralized systems with limited numer of peers at the same time (for example 10)	Suitable for medium (up to 50) teams with central coordination - for example video conferance rooms

As we can see - each solution suits different applications.

1.6. OT vs CRDTs

One very important topic in the matter of conflict resolution and collaboration is data structures algorightms that can be used. Local / remote inserts into data structures need to be **commutative**, **associative** and **idempotent**. These three properties are required to ensure that all replicas of data will be eventually consistent - assuming that all operations (inserts/deletes) propagate properly. There are two main approaches to achieve this - OT (operational transformation) and CRDTs (conflict-free replicated data types).

Historically well known solution is OT - operational transformations. The basic idea behind this approach is to transform operations in such a way that they can be applied in any order and still produce the same result. This approach was used in Google Docs and it is still used in

some applications. However, it has some drawbacks. It is complex to implement and it can lead to conflicts that are difficult to resolve.

CRDTs - conflict-free replicated data types - are a more recent approach to achieve eventual consistency in distributed systems. The basic idea behind CRDTs is to design data structures in such a way that they can be updated independently and still converge to the same state. The algorithms are implemented directly into data structures. This approach is simpler to implement and it can handle conflicts more gracefully. It is used in many modern applications, including Figma, Miro, Notion, Obsidian, Roam Research, Coda and many more.

1.7. Vision of the System

The envisioned system is a desktop application designed for real-time collaborative drawing on a whiteboard. It leverages the Rust programming language for performance and reliability, and utilizes the egui library to provide a modern, responsive user interface. Communication between users is achieved through WebRTC, with the Livekit open-source server facilitating room-based connections. Each room acts as a channel or document, allowing users to join by simply entering a room code or name, without the need for user accounts, authentication, or authorization.

The application employs CRDTs (Conflict-Free Replicated Data Types) from the Automerge library to ensure seamless synchronization of whiteboard state across all participants. This approach enables users to draw and erase simultaneously, with changes instantly reflected for everyone in the room. The cursors of other users are visible, enhancing the collaborative experience. The whiteboard will be represented as a canvas with raster graphics of strokes from users as vector of points with certain width and color.

For real-time, multithreaded communication, the system uses the Tokio library, ensuring efficient handling of concurrent operations. Notably, the application does not persist data by default; once the app is closed, all data is lost unless explicitly exported to a file by the user. The focus is on desktop environments, with no mobile version planned.

1.8. Requirements Specification

1.8.1. Functional Requirements

Below you can see tables with user stories for each actor in the system: user, application, and server.

Table 1.3: Functional Requirements – User Stories (Actor: User) Part 1

ID	**As a...**	**I want to...**	**In order to...**
US-01	User	launch the desktop app and see a blank whiteboard	start drawing immediately without any setup
US-02	User	draw freehand strokes on the whiteboard using a pen tool	express my ideas visually
US-03	User	erase previously drawn strokes	correct mistakes or modify drawings

Table 1.4: Functional Requirements – User Stories (Actor: User) Part 2

ID	As a...	I want to...	In order to...
US-04	User	change the color of the pen	differentiate between elements and add visual clarity
US-05	User	change the thickness (width) of the pen	draw both fine details and bold strokes
US-06	User	clear the entire whiteboard	start drawing from an empty board again
US-07	User	save the current whiteboard state to a file (.crdt/.png)	persist my work for later use
US-08	User	load a previously saved whiteboard document (.crdt/.png)	continue working on a previous drawing
US-09	User	create a new empty document	begin a fresh collaborative session or personal workspace
US-10	User	enter a room name and connect to a collaborative session	work together with other users in real time
US-11	User	disconnect from a collaborative session at any time	leave the room without closing the application
US-12	User	see the cursors of other connected users on the whiteboard	know where collaborators are drawing and coordinate visually
US-13	User	see strokes drawn by other users appear in real time	have a seamless collaborative experience
US-14	User	send chat messages to other participants in the session	communicate textually alongside visual collaboration
US-15	User	be notified when another participant joins or leaves the room	stay aware of who is currently collaborating
US-16	User	join a session without creating an account (only room code needed)	minimize friction and start collaborating instantly

Table 1.5: Functional Requirements – User Stories (Actor: Application, Part 1)

ID	As a...	I want to...	In order to...
AP-01	Application	maintain a local CRDT document (Automerge) representing the whiteboard state	enable offline-capable and conflict-free editing
AP-02	Application	render all strokes stored in the CRDT as raster graphics on a canvas	display the current whiteboard state to the user
AP-03	Application	generate CRDT sync messages when local changes occur	propagate the user's drawing operations to all connected peers
AP-04	Application	receive and apply incoming CRDT sync messages from remote peers	update the local whiteboard with changes made by others
AP-05	Application	establish a WebRTC data channel connection through the LiveKit server	enable real-time, low-latency peer-to-peer communication
AP-06	Application	broadcast the local user's cursor position to all connected peers	allow other users to see where the local user is pointing
AP-07	Application	receive and render remote cursor positions on the whiteboard	display collaborators' cursors with distinguishing color and name
AP-08	Application	run network I/O and CRDT synchronization on separate threads (Tokio)	keep the UI responsive and non-blocking during communication
AP-09	Application	serialize the CRDT document to a byte vector for file export	allow the user to save the full document state
AP-10	Application	deserialize a byte vector from a file into a CRDT document	restore a previously saved whiteboard with full edit history
AP-11	Application	read LiveKit connection credentials from an .env file or environment variables	configure server connectivity without hardcoding secrets

Table 1.6: Functional Requirements – User Stories (Actor: Application, Part 2)

ID	**As a...**	**I want to...**	**In order to...**
AP-12	Application	resolve conflicts between concurrent edits automatically via CRDT merge	guarantee eventual consistency without manual conflict resolution
AP-13	Application	represent each stroke as a vector of points with associated width and color in the CRDT	faithfully reproduce strokes across all replicas
AP-14	Application	provide a modern responsive GUI via the egui library	deliver a native desktop experience on macOS and Windows

Table 1.7: Functional Requirements – User Stories (Actor: Server)

ID	As a...	I want to...	In order to...
SV-01	Server	manage rooms (channels) that users can join by name	group collaborating users into logical sessions
SV-02	Server	relay WebRTC data channel messages between all participants in a room (star topology)	ensure every client receives every peer's sync messages
SV-03	Server	authenticate incoming client connections using API key/secret token validation	prevent unauthorized access to collaboration rooms
SV-04	Server	notify all participants when a new user joins a room	allow clients to initiate CRDT sync with the newcomer
SV-05	Server	notify all participants when a user disconnects from a room	allow clients to remove the disconnected user's cursor and update UI
SV-06	Server	automatically create a room when the first participant connects to a new room name	eliminate the need for manual room provisioning
SV-07	Server	support multiple concurrent rooms with independent participant lists	allow several collaborative sessions to run simultaneously
SV-08	Server	forward data channel packets with minimal latency	support the real-time requirements of collaborative whiteboard synchronization

1.8.2. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system. The FURPS+ model is used to categorize them into Functionality, Usability, Reliability, Performance, and Supportability. The table below presents the non-functional requirements of the collaborative whiteboard application.

Table 1.8: Non-Functional Requirements – FURPS+ Analysis

Category	ID	Description
Usability	NF-01	The user interface shall be simple and intuitive, allowing basic operations (drawing, erasing, connecting to a session) without the need for training.
	NF-02	The system shall clearly display the synchronization status (online/offline) and inform the user about connection state changes.
	NF-03	The editor shall provide visual indication of collaborators' cursors in real time.
	NF-04	The application interface shall be in English.
Reliability	NF-05	The system shall ensure continuity of work in offline mode through local CRDT state, with automatic synchronization upon reconnecting to the session.
	NF-06	The CRDT-based data model shall guarantee that no data is lost during concurrent edits – all user operations shall be preserved after merge.
Performance	NF-07	The synchronization time of changes between users shall not exceed 2 seconds on a connection with bandwidth $\geq$10 Mb/s.
	NF-08	Loading a document from local storage (.crdt file) shall not exceed 1 second.
	NF-09	The application shall support simultaneous editing by at least 3 users on one document without noticeable delays in the interface.
	NF-10	The drawing canvas shall maintain a responsive frame rate ($\geq$30 FPS) during normal drawing operations.
Supportability	NF-11	The system documentation shall include installation and configuration instructions (README) to enable returning to the project after an extended period.
	NF-12	The application shall be buildable and runnable on macOS and windows.
Constraints (+)	NF-13	The application shall be implemented in Rust, using egui for the GUI, Automerge for CRDTs, Tokio for async runtime, and LiveKit for WebRTC signaling.
	NF-14	The application shall not persist any data by default – all state is lost when the app is closed unless explicitly exported by the user.

1.9. Business Cases

Collaboration is neccecary in any business. Companies are build on collaboration and division of work. It is a common effort to build something greater togheter. Therefore having tools that brings real time collaboration is neccecary. The most exiting tools that I think will emarge on the market in coming years are collaborative CAD/CAM applications for engineers. CRDTs might have its best years ahead of them.

1.10. User Interface vision

The assumptions for the user interface are as follows: whiteboard canvas takes up the majority of the screen, with a toolbar in the top bar containing pen and eraser tools, color and thickness selectors, and buttons for creating new documents, saving/loading and menu botton.

The menu side panel on the left can be toggled to show connectivity options, including a text field for entering a room/opening connection panel with information on other users and chat. Cursors of other users are shown as colored circles with their ids next to them at the bottom bar. The design is clean and minimalistic, focusing on ease of use and real-time collaboration.

Although the final version of the user interface differs from the initial vision, the main assumptions were met. The first design is shown in the picture below.

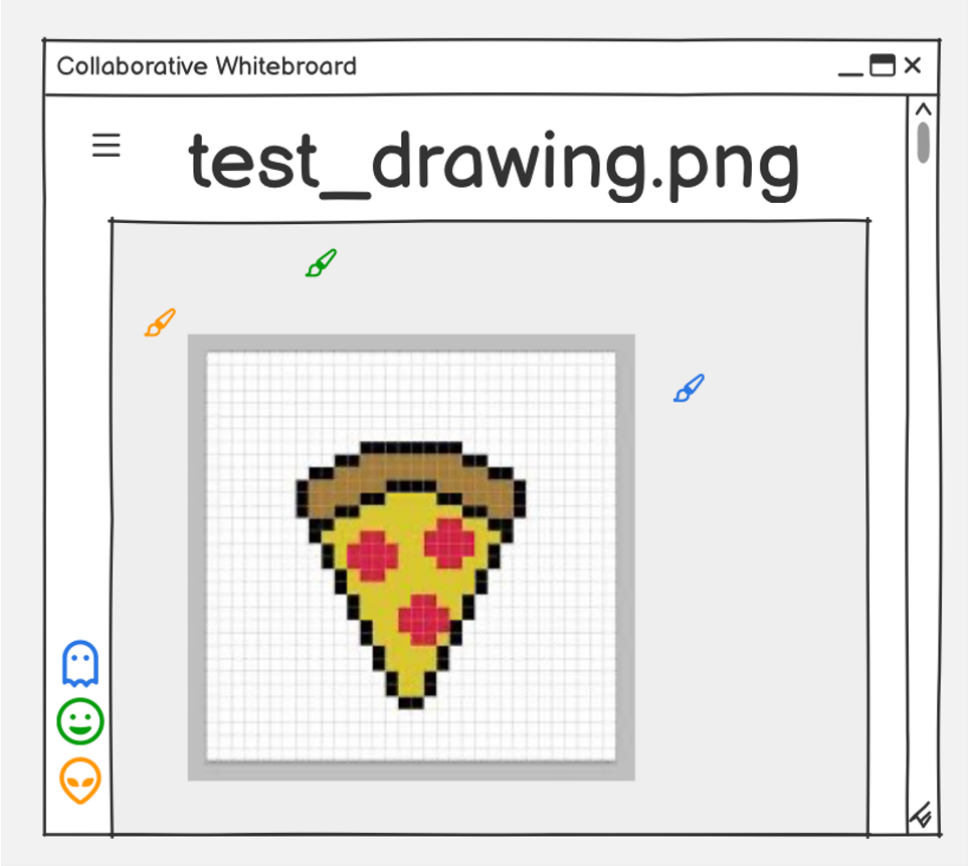

Figure 1.8: Initial vision of the user interface

2. Description of the solution

2.1. System Architecture

As mentioned in the introduction, the system is designed with a star topology architecture, where a central server (open source LiveKit SFU) facilitates communication between clients. Each client maintains a local CRDT document representing the whiteboard state, and synchronization occurs through WebRTC data channels. The architecture is modular, with clear separation of concerns between the user interface (egui), the CRDT data model (Automerge), and the networking layer (LiveKit/Tokio).

The program is devided mostly into three main modules: `ui.rs` which holds most of the application interface state and UX interaction methods, `backend_api.rs` – a module of abstraction which defines connection points of canvas/CRDT and networking, because what user sees on the screen is not just a matrix of pixels from persistent storage, but a result of rendering of the CRDT document.

`automerge_backend.rs` – a module which defines the CRDT document and methods of interaction with it. It is an implementation of `backend_api.rs` with a little bit of extra methods.

2.2. Technology Stack

The stack I decided to implement the solution in is mostly based on my intuition of current trends on software engineering market, therefore I decided to use RUST language - a modern and unusual language, which has gained some popularity in recent years, mostly due to its great implementation of memory safety / ownership of variables. Rust is a language which has a promise of solving approximately 60-70% of vulnerabilities in C/C++ systems, which are memory safety vulnerabilities.

Becasue the application is a desktop application, I decided to use egui library for the user interface. It is a modern and responsive GUI library for Rust, which allows to create native desktop applications with ease. It is also cross-platform, which means that the application can

be run on both macOS and Windows without any changes. Possibly it could also be compiled to webassembly and run in the browser, but it is not a goal of this project.

When deciding on how to implement / use CRDTs in the project, the first idea was to use own implementation of CRDTs, more specifically a RGA - Replicated Growable Array and keep strokes as pixels in a matrix (array of arrays), but during the research, I stumbled upon a very highly regarded and industry wide accepted library developed by Cambride Cumputer Scientist - Martin Kleppmann. The library is Automerge and it implements most of basic data structures as CRDTS. In my case I would use mostly use only few of them, but the library still provides a lot of useful methods for merging, generating sync messages, applying sync messages and so on.

While doing research on how to implement the communication layer, I found out that there is a very good open source project called LiveKit, which is a WebRTC SFU (Selective Forwarding Unit) server. Moreover - the LiveKit provides a Rust SDK. It provides a simple API for creating rooms and managing connections between clients. It also supports data channels, which are perfect for our use case of synchronizing CRDT documents. Therefore, I decided to use LiveKit as the communication layer for the application - both on client side with usage of Rust livekit SDK and on server side with LiveKit SFU as a docker container, which might be easly scaled if needed horizontaly with usage of Kubernetes and load balancers.

The communication needs to be efficient and non-blocking, therefore I decided to use Tokio - a Rust library for asynchronous programming. It provides a runtime for executing asynchronous tasks and allows for efficient handling of I/O operations, which is crucial for real-time communication in a collaborative application.

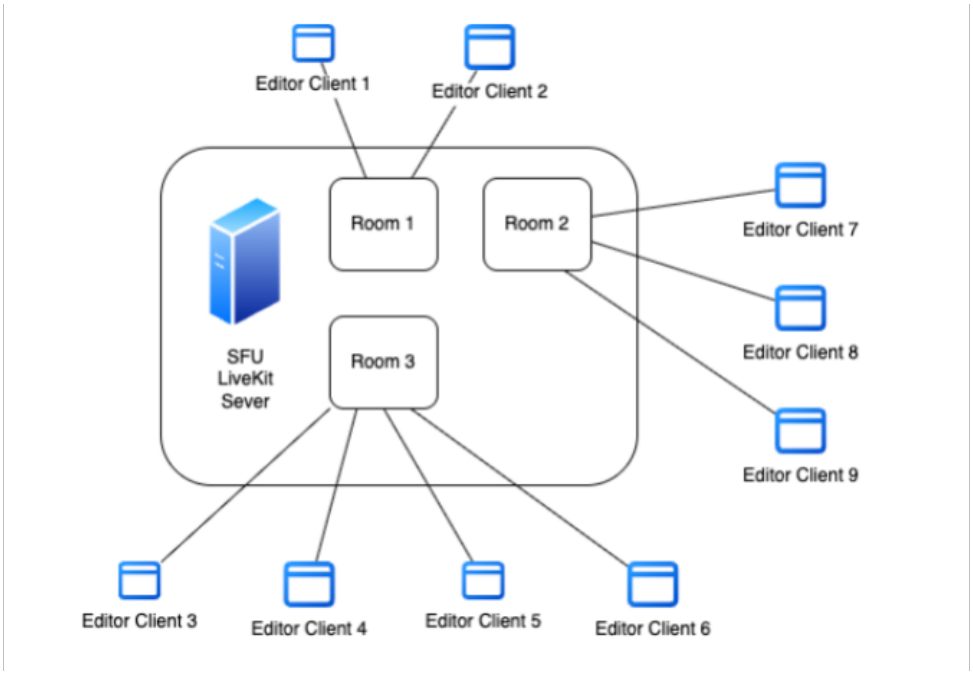

Figure 2.1: Architecture of the envisioned system

2.3. Application Modules

2.3.1. User Interface Modules and Implementation

Most of application state is kept in `AppView` struct. As we can see on the diagram below, it is a struct which holds all information about the state of the application: backend state (`backend`), status bar state (`status`), sidebar state (`SidebatState`), current page state (`page`), whiteboard state (`whiteboard`), LiveKit connection state (`livekit_`), remote cursors state (`remote_cursors`), and so on. It also holds methods for interaction with the user interface, for example methods for rendering the UI (enforced by `egui`), handling user input, and so on. The main method of this struct is `update`, which is called in a loop and is responsible for rendering the UI and handling user input. The outline of the struct is shown in the diagram below. The main point of this struct is to be a single source of truth for the state of the application and to provide methods for interaction with the UI.

Figure 2.2: User Interface Modules 1

Below you can see more precise diagrams of submodules of the user interface module. `Page` enum defines the current page of the application, which can be either `Editor` (whiteboard) or `LiveKit` (connection panel). `SidebarState` struct defines the state of the sidebar, which can be either `visible` or not and have `default_width`. Another important enum is `Tool`, which defines the current tool selected by the user – either `Pen` or `Eraser`.

The `WhiteboardState` struct, shown in the diagram below, encapsulates the entire state of the drawing canvas. It contains the following fields: `image` of type `ColorImage`, which stores the current raster representation of the whiteboard; `texture` of type `Option<TextureHandle>`, which holds the GPU texture used by egui for rendering the canvas; `stroke_color` of type `Color32`, representing the currently selected drawing color; `stroke_width` of type `f32`, defining the thickness of the pen stroke; `current_stroke` of type `Vec<Point>`, which accumulates the points of the stroke currently being drawn by the user before it is committed to the CRDT document; `tool` of type `Tool`, indicating whether the user is currently using the pen or eraser; and `background` of type `Option<ColorImage>`, which optionally stores a background image loaded from a file. This struct is central to the rendering pipeline – on each frame, the `image` is updated based on the strokes stored in the CRDT document, converted into a texture, and displayed on the canvas. The separation of `current_stroke` from the committed strokes allows

for smooth real-time drawing, as the in-progress stroke is rendered locally before being finalized and synchronized with remote peers.

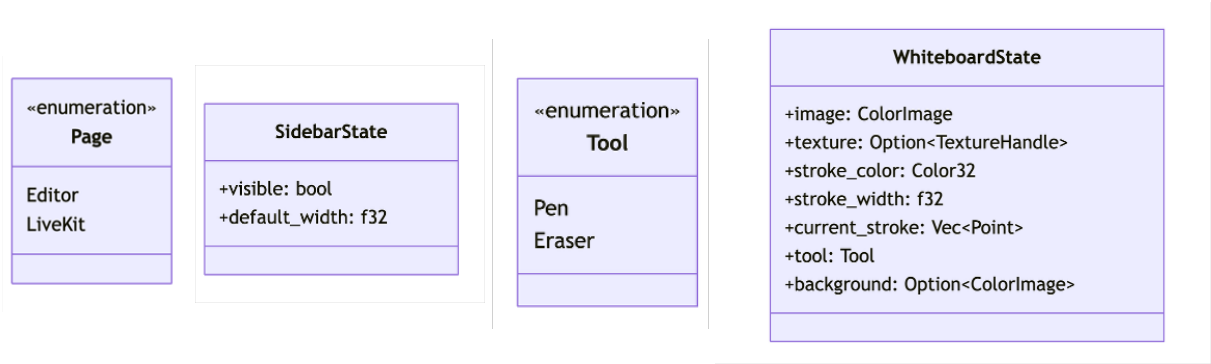

Figure 2.3: User Interface Modules

2.3.2. Backend Module

The backend module defines the boundary between the user interface and the CRDT synchronization logic. At its core lies the `DocBackend` trait (interface), which abstracts all document operations behind a unified API. This separation ensures that the UI layer does not depend on any specific CRDT implementation and can work with any backend that satisfies the trait contract.

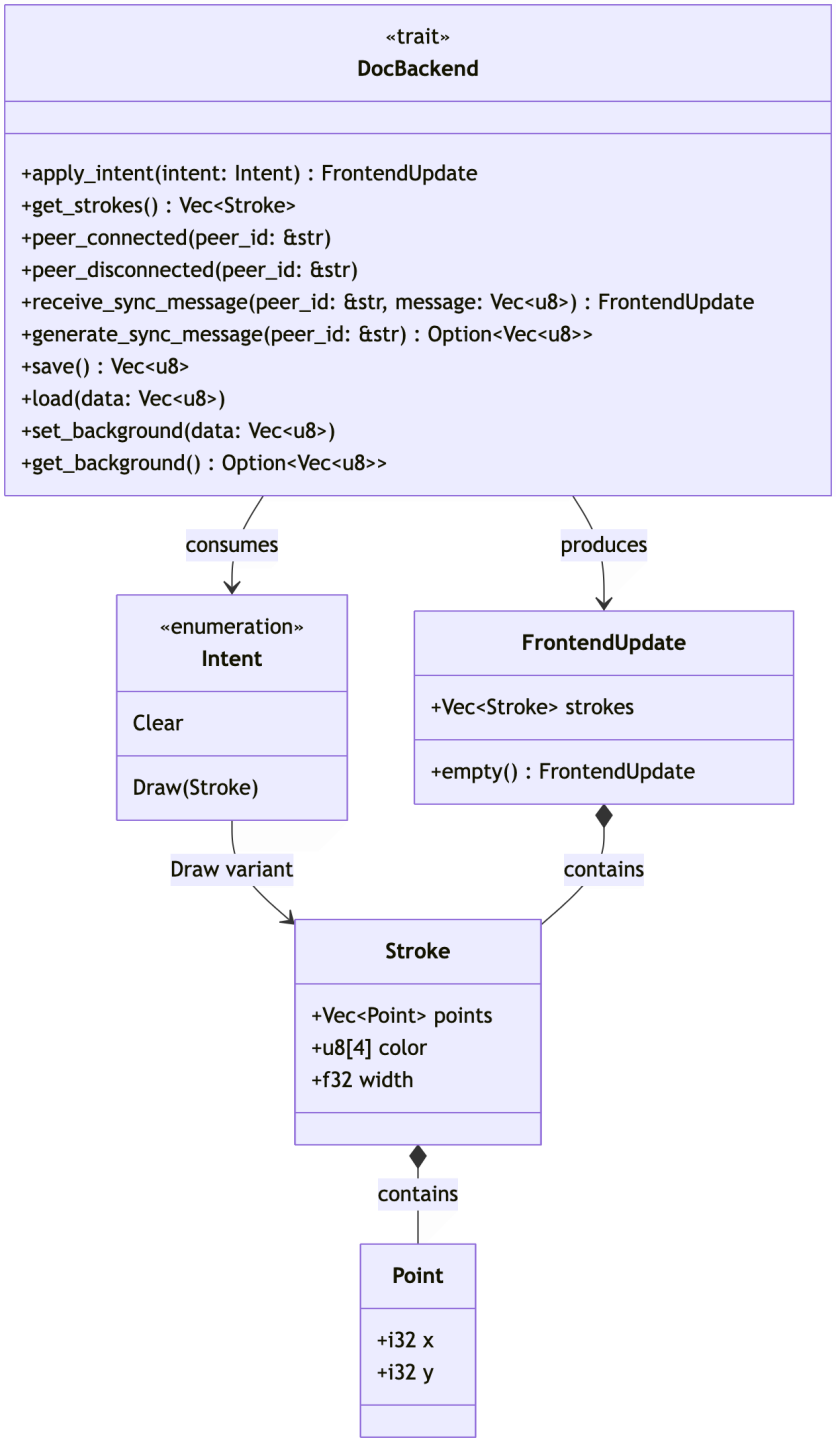

Figure 2.4: Backend Interface

As shown in Figure 2.4, the `DocBackend` interface exposes methods grouped into three categories:

- **Document manipulation** – The `apply_intent` method accepts an `Intent` (either `Draw` or `Clear`) and returns a `FrontendUpdate` containing the current list of strokes to render. The `get_strokes` method allows the UI to retrieve the full set of strokes at any time.

- **Synchronization** – The `peer_connected` and `peer_disconnected` methods notify the backend when remote peers join or leave. The `receive_sync_message` method processes incoming binary sync messages from a peer and returns a `FrontendUpdate` reflecting any changes. On the other hand, `generate_sync_message` produces an outgoing binary message broadcasted to all peers, returning `None` when there is nothing to send.

- **Persistence** – The `save` method serializes the entire document state into a byte vector, while `load` restores the document from previously saved bytes. The 1.0.0+1 version of the applicaiton in ALFA environment enables two file types persistence: .crdt and .png. These methods enable explicit file export and import without requiring any default persistence.

The raster graphics data types are also defined in this module. `Point` represents a 2D coordinate, `Stroke` groups a sequence of points with a color (RGBA) and width, `Intent` encodes user actions, and `FrontendUpdate` carries the resulting list of strokes back to the UI.

This trait-based design makes the backend easily testable and replaceable – one could swap Automerge for a different CRDT library (or even a simple in-memory store for testing) without modifying the user interface code.

2.4. Synchronization Protocol – How consistency is achieved

The synchronization architecture of the collaborative whiteboard can be understood in three layers. First, Section 2.4 introduces the theoretical foundations of the sync protocol – how Automerge ensures that all replicas converge to the same state without a central coordinator. Second, Section 2.5 describes the CRDT data model – the concrete document schema and data structures that the protocol operates on. Finally, Section 2.6 covers the communication layer – the transport infrastructure (LiveKit, WebRTC, Tokio) that carries sync messages between peers. Each layer answers one question in sequence: *how* is consistency achieved, *what* is being synchronized, and *where* do the bytes travel.

2.4.1. State-based vs Operation-based synchronization

Automerge library is positioned as a state-based CRDT because it uses a specialised synchronization protocol to find differences between peers.

- **Sync States**: each peer maintains a `sync::State` for every other connected peer. This acts as a tracker for what the remote peer already knows about the document.

- **Delta updates**: instead of sending the entire document, the backend uses `generate_sync_message()` to produce a compact binary blob containing only the changes (deltas) the specific remote peer has not seen yet.

- **Convergence through merging**: when a peer receives this message, the `receive_sync_message` function decodes the data and automatically merges the remote changes into a local document.

Figure below illustrates the synchronization sequance between two peers - A and B. When A makes a local change, it generates a sync message containing only the new operations that B has not seen yet. This message is sent to B, which applies it to its local document. If B has made concurrent changes, the merge process ensures that both A and B converge to the same final state without conflicts.

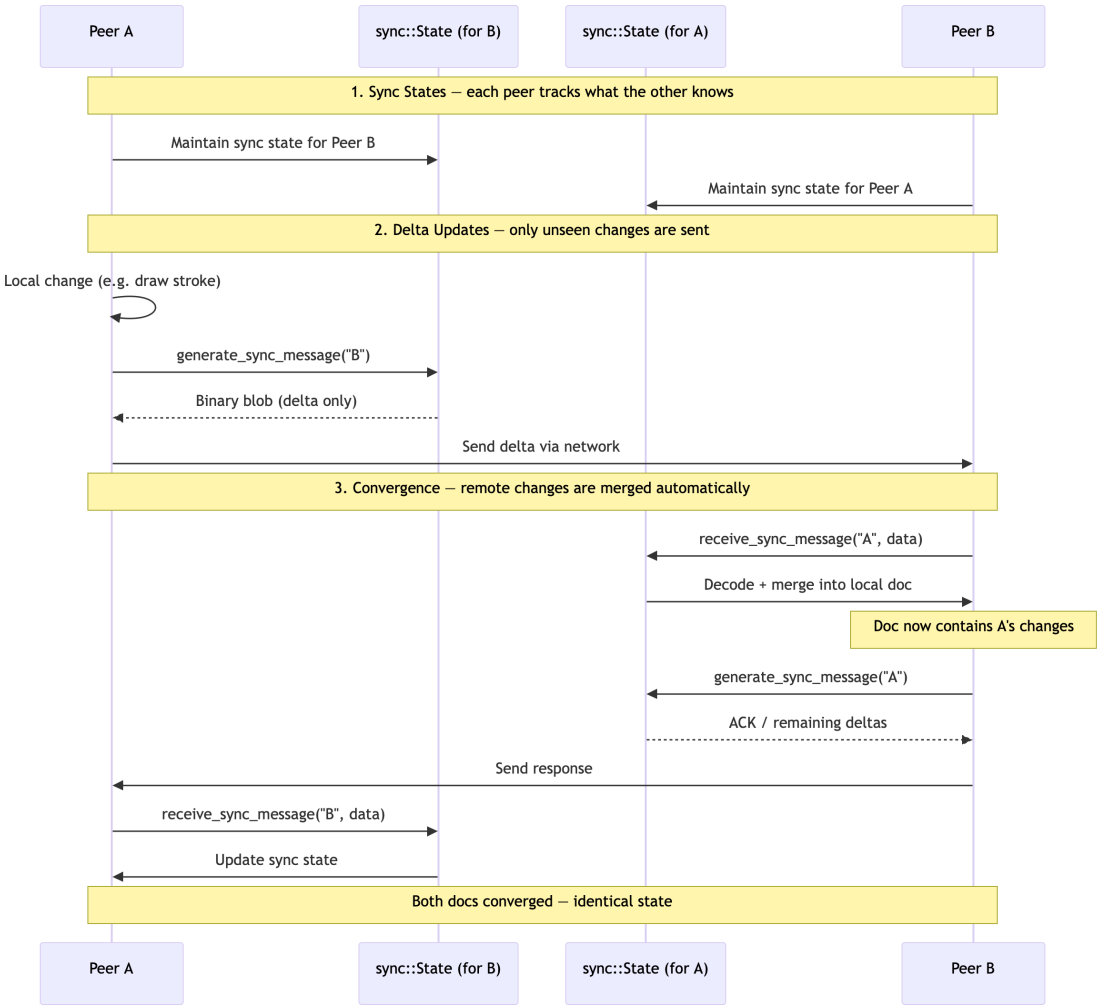

Figure 2.5: Automerge synchronization sequence between two peers

2.4.2. Automerge sync protocol

The synchronization process in Automerge is a state-based handshake designed to bring two peers into the same identical state by exchanging only the necessary data. This is managed through a pair of functions: `generate_sync_message` and `receive_sync_message`.

1. **The Handshake: `generate_sync_message` and `receive_sync_message`**
   The handshake functions as a continuous conversation between peers until they are fully synchronized.

   - `generate_sync_message(peer_id)`: This function asks Automerge to identify what has changed since the last time this specific peer was updated. It uses a peer-peer `sync::State` to track progress. If there are new changes (deltas), they are packed into a binary blob; if the peer is already up to date, it returns `None`.
   - `receive_sync_message(peer_id, message)`: When a peer receives this binary blob, it "unpacks" the changes and merges them into its own document. This function also updates the local `sync::State` for that sender, marking those specific changes as "received" so they aren't requested again.

2. **The role of bloom filters**
   Automerge uses Bloom filters to efficiently track "what the peer already knows" without having to send a massive list of every operation ever performed.

   - Efficient summarization: Instead of sending full history, a peer sends a compact Bloom filter—a probabilistic data structure—that represents the set of operations (changes) it currently holds in its document.
   - Identify gaps: when receiving, the peer compares its own document against this bloom filter, it can quickly identify which operations it has that are not represented in the filter. These missing pieces are then bundled into the next sync message.

3. **Why multiple rounds may be needed**
   While a single change might sync in one go, a full synchronization often requires two to three rounds to reach convergence.

   - **Round 1 (Discovery):** Peer A sends a sync message containing its current "heads" (hashes of recent changes) and a Bloom filter summarizing all the changes it has.
   - **Round 2 (Response):** Peer B receives this, uses the Bloom filter to identify which of its own changes Peer A is missing, and sends those changes back. Peer B also sends its own Bloom filter.

- **Round 3 (Convergence):** Peer A applies the changes from Peer B. It now knows if it is still missing anything based on Peer B's Bloom filter. It sends any remaining changes Peer B might need.
- **Round 4 (Finalization):** Peer B applies those changes. At this point, Peer B generates a final "empty" sync message to confirm to Peer A that they are now in identical states.

Synchronization often requires multiple round-trips to reach convergence, especially if the divergence between peers is significant. The process mimics a conversation where peers exchange what they know until no new information remains. The following Rust code snippet illustrates this iterative negotiation pattern used during a sync session:

```
let mut peer1_state = sync::State::new();
let mut peer2_state = sync::State::new();

loop {
    // 1. Peer 1 generates a message for Peer 2
    let msg1 = doc1.sync().generate_sync_message(&mut peer1_state);

    if let Some(m) = msg1 {
        // Peer 2 applies changes from Peer 1
        doc2.sync().receive_sync_message(&mut peer2_state, m)?;
    }

    // 2. Peer 2 generates a message for Peer 1
    let msg2 = doc2.sync().generate_sync_message(&mut peer2_state);

    if let Some(m) = msg2 {
        // Peer 1 applies changes from Peer 2
        doc1.sync().receive_sync_message(&mut peer1_state, m)?;
    }

    // Stop when both sides have nothing left to send (convergence reached)
    if msg1.is_none() && msg2.is_none() {
        break;
    }
```

```
}
```

This loop ensures that even if one message doesn't carry all necessary changes (due to size limits or dependency resolution), subsequent messages will complete the synchronization.

2.4.3. Conflict resolution and guarantees

In Automerge (and most CRDTs), conflict resolution is governed by the principle of **strong eventual consistency**. This means that if two peers perform different actions on the same piece of data at the same time, they must both arrive at the exact same result without a central server deciding the outcome. Automerge achieves this through deterministic rules embedded directly into the data structure operations.

The "Clear" operation (Delete vs. Edit)

In Automerge, a delete operation (such as clearing a list or removing an element) specifically targets the set of *visible Change IDs* that the peer has observed at the moment the operation is issued.

- **Scenario**: Peer A clears the canvas (deletes all strokes from the list). Simultaneously, Peer B draws a new stroke.
- **Logic**: Peer A's "Clear" command only knows about the strokes that existed at the moment Peer A clicked clear. It has no knowledge of the new stroke Peer B is currently creating, because that stroke has not yet been synchronized.
- **Result**: The new stroke from Peer B *survives*. The old strokes are deleted (since both A and B agree they existed), but the new stroke remains because Peer A's delete operation did not "reach" it. This is the **Add wins** semantics — insertions that a peer has not observed cannot be removed by that peer's concurrent delete.

The "Draw" operation (Concurrent edits to the same property)

If both peers modify the same property concurrently (for example, changing the color of the same stroke at the same time - hypothetically, because 1.0.0 version of application does not allow such operations), Automerge must deterministically choose which value to display.

- **Scenario**: Peer A sets a stroke color to Red. Peer B sets the same stroke color to Blue.
- **Logic**: Automerge performs a deterministic **Last Write Wins (LWW)** tie-break based on a lexicographical comparison of the internal Change IDs (hashes).

- **Result**: If Peer A's change hash is `0xabc...` and Peer B's is `0xdef...`, Peer B wins because its hash is lexicographically higher. Crucially, both peers will see the same winner. No data is actually lost in the operation history — Automerge preserves the full history — but only one value is displayed as the "current" state.

The following table summarizes the conflict resolution strategies:

Table 2.1: Conflict resolution strategies in Automerge

Action	**Conflict Type**	**Resolution**
Clear	Add vs. Remove	Add wins. Deletes only affect elements the peer observed at the time of deletion.
Draw	Value vs. Value	Deterministic tie-break. The change with the higher lexicographical hash becomes the visible value.

Implications for the whiteboard application

In the implemented Rust application, strokes are stored in an Automerge `List`. Calling `splice` to clear the list or `insert` to add a stroke is inherently safe under concurrent usage. If one user clears the whiteboard while another draws a new stroke, that new stroke will appear on both screens after the sync loop finishes, while the old "cleared" strokes will vanish. This prevents the "lost update" problem where one user's work is accidentally wiped out by another's cleanup action.

If the application-level intent is that "Clear" should mean "wipe everything, including things I have not yet seen," this must be handled at the application layer — for example, by storing a `last_cleared_timestamp` and filtering out any strokes created before that time. In the current implementation, the simpler CRDT-native semantics (add wins) are used, which is the safer and more predictable default for collaborative drawing.

2.4.4. Initial synchronization of a new peer

A critical aspect of the collaborative experience is the ability for a new user to join an existing session and seamlessly acquire the current state of the whiteboard. When a new peer joins a room (mid-session), they start with an empty local document (or a stale version if they were previously connected). The synchronization protocol handles this case naturally without special "initialization" logic.

Upon connection, the existing peers in the room detect the new participant (via LiveKit events) and initiate the standard sync handshake described in Section 2.4.2. Because the new peer's knowledge state is zero (empty Bloom filter), the first sync message generated by an existing peer will contain the entire history of operations needed to reconstruct the current document state. The new peer receives this "full" update, applies it, and instantly converges to the same state as the rest of the group. This process ensures that late-joining users are never out of sync and have access to the full drawing history, allowing them to undo or modify strokes made before they arrived.

2.5. CRDT Data Model – What is being synchronized?

Having described *how* the synchronization protocol achieves consistency, this section focuses on *what* is being synchronized — the concrete data structures stored inside the Automerge document. The data model defines how user-visible whiteboard elements (strokes, background) are represented as CRDT objects, how user actions map to CRDT operations, and how the entire document can be persisted to and restored from disk.

2.5.1. Document schema

The Automerge document follows a simple, flat schema with two top-level keys stored under the document root (`ROOT`):

```
ROOT
 +-- "strokes"    : List<String>       // JSON-serialized strokes
 +-- "background" : Bytes              // optional background image
```

- `strokes` — an Automerge `List` where each element is a JSON string representing a single `Stroke` object. The list is created lazily: if it does not exist when a stroke is drawn, the backend creates it with `put_object(ROOT, "strokes", ObjType::List)`.
- `background` — an optional `Bytes` scalar stored directly at the root. It holds the raw binary data (e.g., PNG) of a background image, if one has been loaded by the user.

Why JSON strings inside a List? An alternative design would be to use native Automerge maps and nested lists to represent each stroke's points, color, and width. However, storing strokes as opaque JSON strings offers several practical advantages:

1. **Simplicity**: Serialization and deserialization are handled by `serde_json`, avoiding the complexity of managing deeply nested Automerge objects with multiple object IDs.

2. **Atomicity**: Each stroke is inserted or removed as a single list element. There is no risk of partially synchronized strokes (e.g., points arriving without their color).

3. **Performance**: Fewer CRDT metadata objects are created, reducing memory overhead and sync message size.

4. **Trade-off**: The downside is that individual stroke properties (e.g., color) cannot be edited in-place without replacing the entire JSON string. For the whiteboard use case, where strokes are immutable once drawn, this trade-off is acceptable. A notable consequence of this design is the eraser implementation: rather than removing or modifying existing strokes, the eraser tool works by drawing new strokes with a white color (`[255, 255, 255, 255]`), effectively painting over previous content. This approach is discussed further in the "Eraser implementation" paragraph below.

2.5.2. Stroke representation

The data types that model a stroke are defined in the `backend_api` module. A `Stroke` consists of three fields:

```
pub struct Point {
    pub x: i32,
    pub y: i32,
}

pub struct Stroke {
    pub points: Vec<Point>,    // ordered sequence of coordinates
    pub color: [u8; 4],        // RGBA color
    pub width: f32,            // line thickness in pixels
}
```

Why a vector of points rather than pixels? Two approaches exist for representing drawn content: storing raw pixel data (a bitmap) or storing geometric primitives (vectors of points). The vector approach was chosen for several reasons:

- **Compactness**: A stroke of 200 sampled points requires approximately $200 \times 8 = 1600$ bytes

plus color and width metadata. A rasterized equivalent on a 1920 × 1080 canvas would require megabytes.

- **Resolution independence**: Strokes can be re-rendered at any zoom level or canvas size without loss of quality.
- **CRDT friendliness**: Each stroke is a self-contained, serializable unit that can be inserted into or removed from the Automerge list atomically. Bitmap-based approaches would require pixel-level conflict resolution, which is far more complex.
- **Undo granularity**: Removing a stroke means deleting one list element rather than restoring thousands of pixels.

2.5.3. Intent abstraction

User actions are abstracted through the `Intent` enum, which decouples the UI from the CRDT operations:

```
pub enum Intent {
    Draw(Stroke),   // add a new stroke to the document
    Clear,          // remove all strokes from the document
}
```

Each intent maps directly to specific Automerge CRDT operations:

Table 2.2: Mapping of user intents to Automerge CRDT operations

Intent	**Action**	**Automerge operation**
`Draw`	Serialize stroke to JSON, append to end of list	`insert(&list_id, len, ScalarValue::Str(json))`
`Draw` (eraser)	Serialize white stroke to JSON, append to end of list	`insert(&list_id, len, ScalarValue::Str(json))`
`Clear`	Remove all elements from the strokes list	`splice(&list_id, 0, len, empty)`

The sequence of operations triggered by `Intent::Draw` is illustrated in Figure 2.6. The backend first serializes the `Stroke` to JSON, then looks up (or creates) the `"strokes"` list in the document, inserts the JSON string at the end, and finally reads back all strokes to produce a `FrontendUpdate` for the UI.

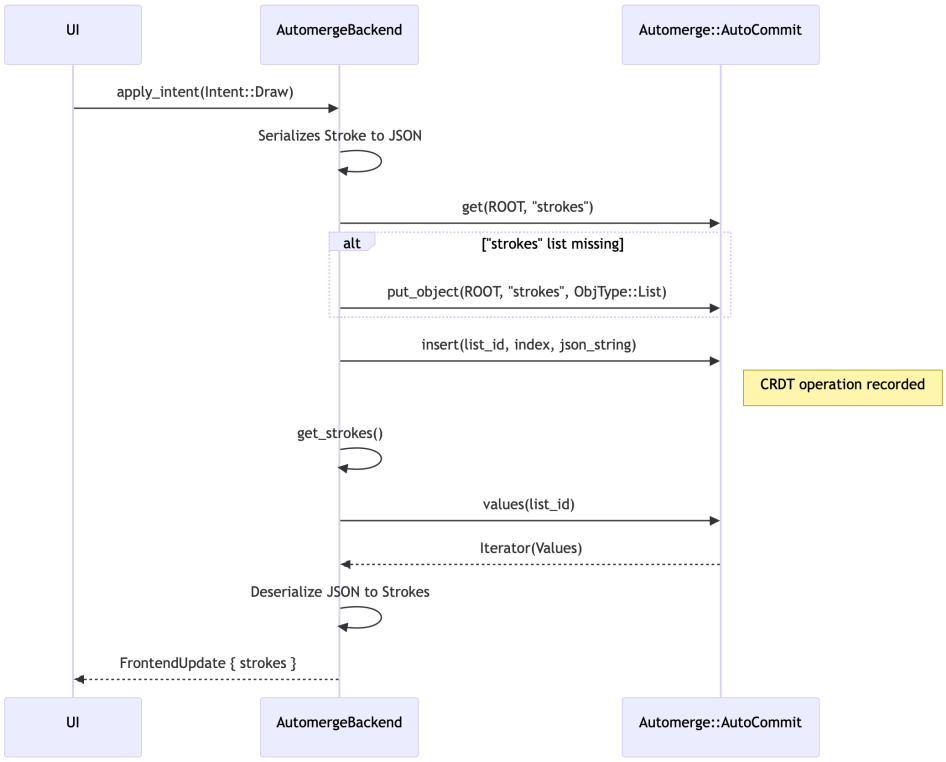

Figure 2.6: Sequence diagram of a local `Intent::Draw` operation

For `Intent::Clear`, the backend retrieves the list and calls `splice` with a delete count equal to the list length and an empty iterator for replacements. This removes all elements in a single CRDT operation. As discussed in Section 2.4.3, this delete only affects strokes that the local peer has observed — any concurrently inserted strokes from remote peers will survive the clear.

Eraser implementation Although the user interface presents the eraser as a distinct tool alongside the pen, the eraser does not correspond to a separate `Intent` variant. Instead, the eraser is implemented entirely at the UI layer by substituting the user-selected color with opaque white (`Color32::WHITE`, i.e. RGBA `[255, 255, 255, 255]`). When the user releases the eraser stroke, the UI commits it through the same `Intent::Draw` path as a regular pen stroke. From the CRDT's perspective, an eraser action is indistinguishable from drawing a white line — it is simply another entry appended to the `"strokes"` list.

This design has several implications:

- **Simplicity**: No additional CRDT operation or intent type is needed. The backend remains unaware of the eraser concept entirely, keeping the synchronization logic minimal.
- **Additive nature**: Eraser strokes are additive — they increase the size of the stroke list rather than removing elements. Over extended drawing sessions, this can lead to a growing

document even when the user is "erasing" content.

- **Background limitation**: Because the eraser paints white pixels, it only produces the expected visual effect on a white canvas. If a background image has been loaded, the eraser will paint opaque white over the background rather than restoring the underlying image.
- **No structural removal**: Individual strokes beneath the eraser path are not removed from the CRDT document. They remain in the operation history and continue to be synchronized with peers. The "erasure" is purely visual — the white stroke is rendered on top during the rasterization pass, hiding the strokes underneath.

An alternative approach would be to introduce an `Intent::Erase` variant that performs hit-testing against existing strokes and removes intersecting entries from the Automerge list using `splice`. This would produce true structural deletion and reduce document growth. However, it would require geometric intersection logic (testing whether the eraser brush overlaps each stroke's point sequence) and introduce more complex CRDT operations. For the current version of the application, the simpler white-stroke approach was chosen as a conscious trade-off favoring implementation simplicity over document efficiency.

2.5.4. File persistence

The application does not persist data by default. However, the user can explicitly save and load documents through the `save` and `load` methods of the `DocBackend` trait. Two export formats are supported:

- `.crdt` **(full state)**: The `save()` method calls `self.doc.save()`, which serializes the entire Automerge document — including all operation history, actor IDs, and the current state — into a compact binary format. Loading this file with `AutoCommit::load(&data)` fully reconstructs the document, preserving the ability to merge with other peers' documents. After loading, all sync states are cleared because the relationship with previously connected peers is no longer valid.
- `.png` **(raster snapshot)**: The UI layer can also export the current canvas as a PNG image. This is a one-way export — the raster image cannot be loaded back into the CRDT — but it is useful for sharing the whiteboard content outside the application.

```
// Saving: serialize the full CRDT document to bytes
fn save(&mut self) -> Vec<u8> {
```

```
    self.doc.save()
}

// Loading: reconstruct document from bytes, reset sync states
fn load(&mut self, data: Vec<u8>) {
    if let Ok(doc) = AutoCommit::load(&data) {
        self.doc = doc;
        self.sync_states.clear();
    }
}
```

The `.crdt` format preserves the full operation history, which means that a loaded document can be taken into a new collaborative session and merged with other peers' changes seamlessly. This is a direct benefit of the CRDT architecture — the document carries its own synchronization context.

2.6. Communication Layer - Where the messages travel ?

In previous sections, we have established *how* the synchronization protocol achieves consistency and *what* data structures are being synchronized. This section focuses on the *communication layer* — the transport infrastructure that carries sync messages and other real-time updates between peers in a collaborative session.

2.6.1. LiveKit and WebRTC data channels

The application uses LiveKit as the underlying real-time communication platform. LiveKit provides a scalable SFU (Selective Forwarding Unit) architecture that allows multiple peers to connect to a shared room and exchange media and data streams efficiently. It is an open-source alternative to services like Twilio or Agora, offering low-latency connectivity and built-in support for WebRTC data channels, which are ideal for real-time synchronization of CRDT messages and cursor positions in a collaborative whiteboard application. This solution uses a start topology where each peer connects to the LiveKit server, which forwards messages to all other participants in the room. The pros and cons of this approach were discussed in the introduction chapter.

WebRTC data channels provide a low-latency, peer-to-peer communication mechanism that is ideal for transmitting arbitrary binary data between clients in real time. Unlike traditional client-server HTTP APIs, data channels are bidirectional and support both reliable and unreliable

delivery modes, making them suitable for collaborative applications where timely updates are critical.

In this system, each client establishes a WebRTC data channel through the LiveKit server upon joining a room. All CRDT synchronization messages, cursor updates, and chat messages are serialized into binary form and sent over this channel.

LiveKit abstracts the complexity of WebRTC signaling and connection management, so the application code can focus on encoding and decoding messages for synchronization. This approach provides a robust foundation for building scalable, responsive collaborative tools.

2.6.2. Thread modeling

Application in its sourcecode uses Tokio runtime library which enables convinient usage of threads in application. The UI runs on the main thread, while all network communication and CRDT synchronization logic is executed on a separate background thread managed by Tokio. T his separation ensures that the user interface remains responsive and does not block while waiting for network messages or performing potentially expensive CRDT operations.

In rust communication between threads is handled using MPSC (multi-producer, single-consumer) one-way channels. In this application, the UI thread sends user intents (e.g., drawing a stroke) to the backend thread via an MPSC channel. The backend processes these intents, updates the CRDT document, and generates frontend updates (e.g., the new list of strokes) that are sent back to the UI thread through another MPSC channel. This asynchronous message-passing architecture allows for smooth interaction without freezing the UI, even during intensive synchronization tasks. But before this happens, client needs to connect to the LiveKit room, which is also done on the background thread to avoid blocking the UI during connection setup. Below you can see a sequence diagram illustrating connection of client to LiveKit room.

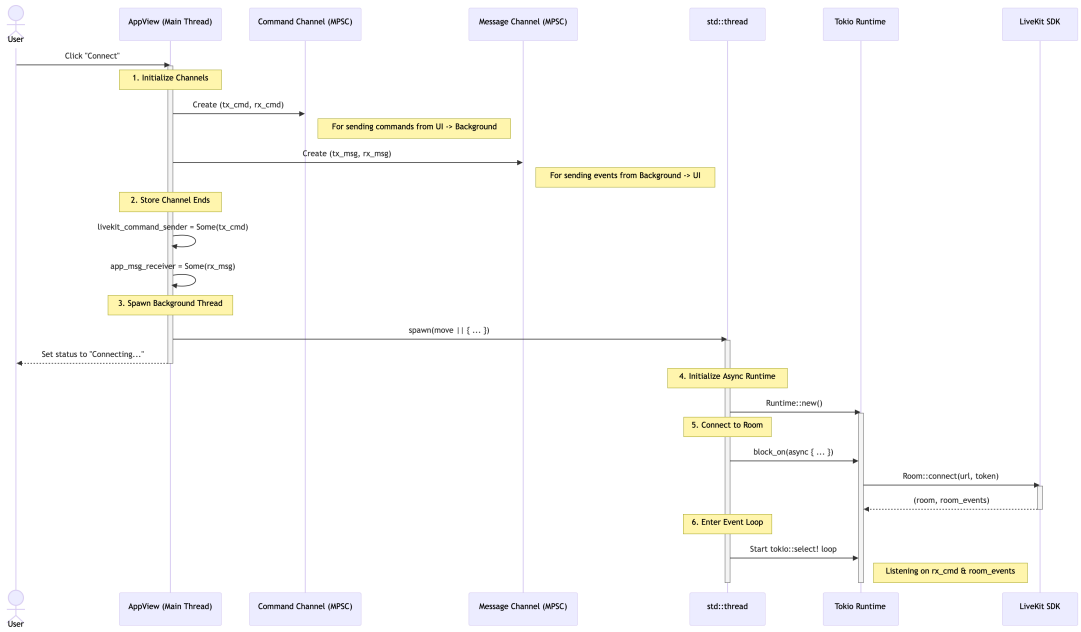

Figure 2.7: Threaded connection sequence: UI thread and Tokio background thread during LiveKit room connection

The diagram above is very complcated, but it illustrates the key steps involved in connecting to a LiveKit room without blocking the UI thread. The user initiates the connection by clicking a button, which initializes MPSC channels in both directions. Then thread is spawned. The proccess inside the thread conencts to the room and starts a listener to events from livekit data channel.

2.6.3. Message flow and chunking

how sync messages are published/broadcasted, the chunking mechanism for messages exceeding 14KB - livekit limit, reassembky in the receiving side

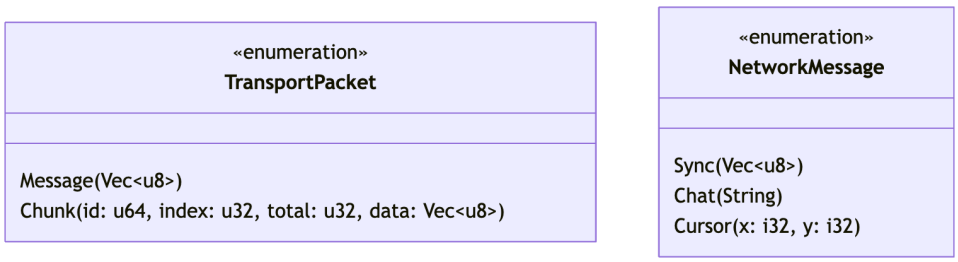

Figure 2.8: Classes of connectivity in collaborative applications

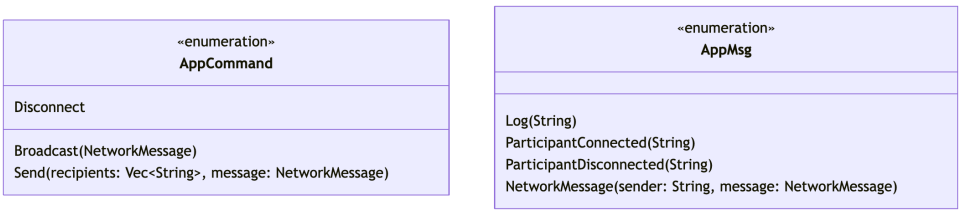

Figure 2.9: Classes of connectivity in collaborative applications

2.6.4. Cursor synchronization

cursor positions are broadcasted separetly from CRDT sync (lightweight, high-frequancy) not part of a CRDT document

final screenshots, comparison etc

3. Analysis of the solution

4. Conclusions

Bibliography

[1] A. Author, *Title of a book*, Publisher, year, page–page.

[2] J. Bobkowski, S. Dobkowski, Title of an article, *Magazine X, No. 7*, year, PAGE–PAGE.

[3] C. Brink, Power structures, *Algebra Universalis 30(2)*, 1993, 177–216.

[4] F. Burris, H. P. Sankappanavar, *A Course of Universal Algebra*, Springer-Verlag, New York, 1981.

List of symbols and abbreviations

nzw.	nadzwyczajny
*	star operator
~	tilde

If you don't need it, delete it.

List of Figures

1.1 Simple client-server architecture of early internet era applications 12
1.2 More complicated architecture of modern mobile/web applications 12
1.3 Typical modules of modern cloud-based applications 13
1.4 Typical sequance of interaction with a user in a modern cloud-based applications 14
1.5 Typical sequance of interaction with a user in a local-first applications 15
1.6 Cloud vs Local comparison 16
1.7 Comparison of Client-Server and Peer-to-Peer (P2P) Architectures 17
1.8 Initial vision of the user interface 27

2.1 Architecture of the envisioned system 30
2.2 User Interface Modules 1 31
2.3 User Interface Modules 32
2.4 Backend Interface 33
2.5 Automerge synchronization sequence between two peers 35
2.6 Sequence diagram of a local `Intent::Draw` operation 43
2.7 Threaded connection sequence: UI thread and Tokio background thread during LiveKit room connection 47
2.8 Classes of connectivity in collaborative applications 47
2.9 Classes of connectivity in collaborative applications 48

If you don't need it, delete it.

Spis tabel

1.1 Use cases: Suitable and Unsuitable for Local-First Approach 16
1.2 Comparison of Peer-to-Peer and Star Topology Architectures 18
1.3 Functional Requirements – User Stories (Actor: User) Part 1 20
1.4 Functional Requirements – User Stories (Actor: User) Part 2 21
1.5 Functional Requirements – User Stories (Actor: Application, Part 1) 22
1.6 Functional Requirements – User Stories (Actor: Application, Part 2) 23
1.7 Functional Requirements – User Stories (Actor: Server) 24
1.8 Non-Functional Requirements – FURPS+ Analysis 25

2.1 Conflict resolution strategies in Automerge 39
2.2 Mapping of user intents to Automerge CRDT operations 42

If you don't need it, delete it.

List of appendices

1. Appendix 1
2. Appendix 2
3. In case of no appendices, delete this part.